



Enunciado de Projeto

Simulador de Elevadores em Prédio Virtual

Conteúdo

1	Introdução	1
2	Regras	1
3	Especificação Funcional e Técnica do Simulador	3
4	Documentação e Entregas	5
5	Tabela de Cotações e Penalizações	7

1 Introdução

O objetivo deste projeto é desenvolver uma aplicação em Java que simule o funcionamento de elevadores num edifício virtual, aplicando os princípios da programação orientada a objetos, padrões de desenho de software e técnicas de refactoring. A aplicação deverá incluir uma interface gráfica e uma interface em modo consola que permita visualizar o estado dos elevadores, os pisos e os passageiros, sem necessidade de interação direta do utilizador durante a simulação.

O projeto será desenvolvido em três fases:

- **Fase 1 (Milestone):** Estrutura base do projeto e modelação inicial do problema.
- **Fase 2 (Projeto Funcional):** Implementação completa das principais funcionalidades, incluindo dois modos de visualização — um em consola e outro com JavaFX.
- **Fase 3 (Entrega Final):** Refactoring e otimização do código desenvolvido anteriormente, bem como implementação de uma funcionalidade adicional que será divulgada posteriormente.

2 Regras

O não cumprimento das regras a seguir descritas implica uma penalização na nota do trabalho prático. Se ocorrer alguma situação não prevista nas regras a seguir expostas, essa ocorrência deverá ser comunicada ao respetivo docente de laboratório de PA e/ou à RUC.

2.1 Regras de Elaboração e Submissão

- a) O projeto deverá ser elaborado por 3 (três) estudantes do mesmo docente de laboratório. O desvio desta regra terá de ter o consentimento expresso da RUC.
- b) O projeto está dividido em 3 fases, com entregas específicas para cada uma delas, que serão submetidos no Moodle até à data-limite e hora respetiva. O conteúdo de cada entrega deve seguir o descrito na seção 4.3 e que se resume na tabela abaixo.

Fase	Data-limite	Resumo dos Entregáveis
1 – Milestone	10 nov 2025, 9h00	Implementação do Modelo + Testes + Mockup + README (com planeamento de tarefas)
2 – Projeto Funcional	9 dez 2025, 9h00	Aplicação funcional em JavaFX
3 – Final	9 jan 2026, 23h00	Aplicação final + JavaDoc + Relatório

- c) No rosto do relatório e nos ficheiros de implementação deverá constar o número, nome e turma dos autores, bem como o nome do docente a que se destina. Para tal, deverão criar um arquivo ZIP com o nome: **<númeroLeader>_<nomeLeader>.zip**, onde colocarão o deliverable respetivo à entrega. O leader do grupo será responsável pela submissão no Moodle até à data-limite.
- d) As datas das discussões serão publicadas após a entrega dos trabalhos.
- e) O desenvolvimento e versionamento do projeto deverá ser efetuado através de um repositório Git, criado automaticamente após a aceitação do *assignment* do GitHub Classroom (ver instruções no Moodle). O repositório inicial contém um projeto IntelliJ IDEA configurado com o sistema de build Maven.

2.2 Regras de Avaliação

- a) A nota do Projeto será atribuída individualmente a cada elemento do grupo após a discussão. As discussões poderão ser orais e/ou escritas. As orais poderão realizar-se com todos os elementos em simultâneo ou individualmente. A nota da discussão tem peso de 35% na nota final e exige nota mínima de 9,5 valores para aprovação.
- b) Os commits no repositório Git individual do grupo serão considerados na avaliação individual.
- c) A apresentação de relatórios ou implementações plagiadas leva à atribuição imediata de nota zero a todos os trabalhos envolvidos, quer tenham sido o original ou a cópia.
- d) A apresentação de implementações com origem em ferramentas de IA não referenciadas (no código e no relatório) leva à atribuição imediata de nota zero, pois o objetivo é avaliar as competências de programação dos estudantes e não as das ferramentas. **Se algum excerto de código tiver sido gerado por IA, deve ser claramente identificado no código-fonte.**
- e) A tabela com as cotações detalhadas encontra-se no no Cap 5 .

3 Especificação Funcional e Técnica do Simulador

3.1 Descrição geral do simulador

Pretende-se ter uma aplicação que permita a simulação de elevadores num prédio virtual, considerando as seguintes características:

- O número de pisos e de elevadores é definido no arranque do simulador;
- Cada piso dispõe de um botão de chamada comum a todos os elevadores;
- Cada elevador possui uma porta que abre em cada operação de embarque/desembarque e fecha posteriormente;
- Os passageiros são representados graficamente e possuem um piso de destino;
- Existem três tipos de passageiros: Crianças, Adultos e Idosos - com prioridades de embarque na seguinte ordem: Idosos > Crianças > Adultos;
- Cada elevador tem uma lotação máxima (configurável) e mantém a lista dos pisos de destino dos passageiros;
- Durante a simulação, o surgimento de novos passageiros é probabilístico: existe uma determinada probabilidade de aparecerem passageiros em pisos aleatórios.

O simulador deverá apresentar estatísticas, como distância percorrida por cada elevador, tempo de inatividade e tempo médio de espera dos passageiros.

A simulação é automática: as pessoas são colocadas em pisos aleatórios e recebem pisos de destino também atribuídos aleatoriamente. O controlo de múltiplos elevadores deve ser cuidadosamente modelado, considerando fatores como direção, distância e estado atual de cada elevador.

💡 A lógica requerida para controlar elevadores é mais complexa do que aparenta, particularmente no que toca a responder a chamadas quando existem múltiplos elevadores por onde escolher. Deve-se escolher o que está mais perto? Provavelmente não, se se estiver a movimentar no sentido oposto ao piso de chamada. E se um elevador parado está a um piso de distância, mas existe um elevador, a dois pisos de distância, a se movimentar na direção do piso de chamada? Toda a gestão de pedidos pode tornar-se bastante complexa. É dado ênfase ao facto de o objetivo principal do projeto ser o de modelar o problema e só depois otimizar o funcionamento dos elevadores.

3.2 Requisitos Funcionais e Técnicos

O simulador deverá ser implementado em Java, utilizando programação orientada por objetos, tipos abstratos de dados (ADTs) e padrões de desenho de software. A qualidade do projeto será avaliada com base na clareza do modelo de dados, na correção funcional e na aplicação adequada dos conceitos anteriores.

O simulador deverá permitir a utilização simultânea de dois ou mais elevadores e suportar um número variável de pisos (mínimo de três).

Aspetos de implementação obrigatórios:

- Utilização de diferentes ADTs (coleções) para representar filas de espera, passageiros em cada elevador e pedidos pendentes;

- Separação clara entre as classes de modelo (lógica da simulação) e visualização (interface gráfica – consola/JavaFX);
- A interface gráfica deve permitir visualizar o estado atual dos elevadores, pisos e passageiros, sem necessidade de animação;
- Os algoritmos de gestão dos elevadores devem implementar o padrão de desenho Strategy, permitindo alterar o algoritmo de gestão dos elevadores em tempo de execução;
- A criação de diferentes tipos de passageiros deve seguir um padrão de desenho criacional;
- Qualquer ADT adicional, exceto listas e dicionários, deverá ser implementado pelos alunos com base nas técnicas estudadas na unidade curricular.

A simulação decorre em passos discretos de tempo t . A duração de cada passo – correspondente à transição de t para $t+1$ – é constante e expressa em milissegundos. Este valor deve ser configurável, permitindo ajustar a velocidade da simulação.

Em cada instante t , cada elevador encontra-se num único **estado** definido pelo diagrama de estados apresentado na Figura 1 e que deverá ser modelado com o padrão de desenho State. Cada transição de estado demora o mesmo tempo, o que garante que os elevadores estão sempre num piso específico, e nunca entre pisos.

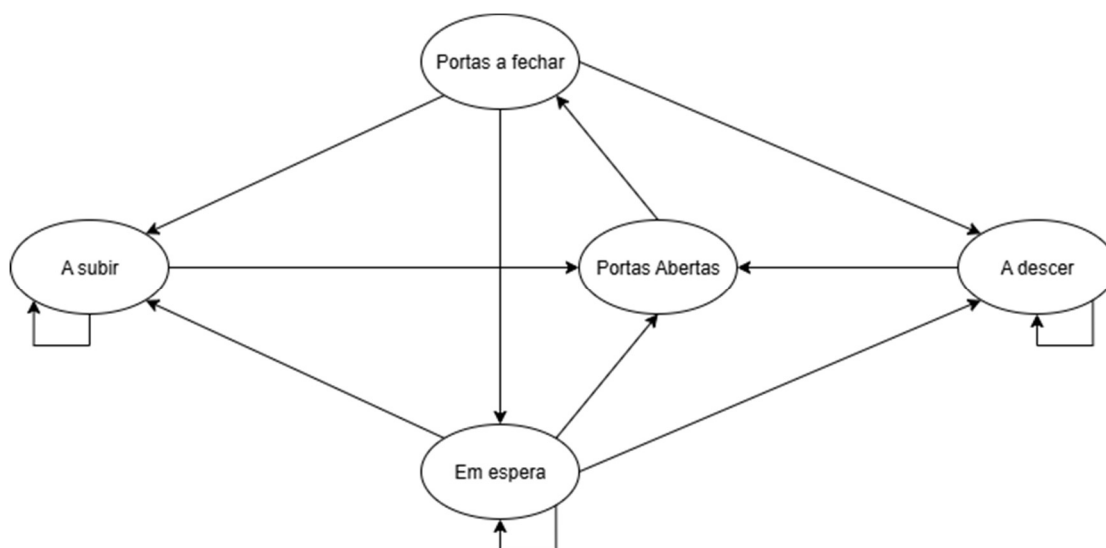


Figura 1- Diagrama de estados de um elevador.

Ao longo da simulação (i.e., a cada instante t), podem surgir novos pedidos de transporte, correspondendo ao aparecimento de passageiros em pisos aleatórios. O surgimento desses pedidos deve ser determinado de forma probabilística, com uma probabilidade configurável e, opcionalmente, um número máximo de passageiros gerados em cada evento.

Pelo diagrama e especificação do problema, deverá conseguir derivar as regras de transição de estados. Salvaguarda-se, no entanto, que o embarque e desembarque de passageiros é feito durante o estado de “Portas Abertas”.

4 Documentação e Entregas

4.1 Documentação Javadoc

Todo o código deve ser documentado utilizando Javadoc. A documentação deverá ser gerada em formato HTML e entregue junto com o projeto, colocada na pasta `javadoc` na raiz do ficheiro ZIP submetido. A documentação deverá cobrir todas as classes públicas, interfaces e métodos relevantes, incluindo:

- Descrição sumária da responsabilidade da classe/interface;
- Javadoc para todos os construtores públicos, métodos públicos e protegidos, com descrição dos parâmetros (`@param`) e do valor de retorno (`@return`) quando aplicável;
- Excepções lançadas documentadas com `@throws`;
- Exemplos de uso breves nos comentários (`@example` ou bloco de `<pre>`), quando úteis para clarificar o comportamento.

Critérios mínimos de avaliação da documentação Javadoc:

- Cobertura: todas as classes e métodos públicos têm documentação;
- Qualidade: descrições claras e concisas que expliquem a finalidade e o contrato dos elementos documentados;
- Exemplos e notas onde o comportamento possa ser ambíguo;

4.2 Relatório

O relatório a entregar na fase 3 deverá conter, além da capa, as secções enumeradas a seguir. Recomenda-se um tamanho total entre 8 e 20 páginas (excluindo anexos), mas a clareza e a concisão são mais importantes que o número de páginas.

- o. Capa e Identificação do Grupo (assinalando o membro leader)
1. Tipos Abstratos de Dados:
 - A apresentação do(s) ADT(s) implementado(s) – descrição da interface Java e da implementação obtida;
2. Diagrama de classes:
 - Inclua diagramas UML legíveis. Diagramas ilegíveis serão penalizados com nota zero nessa secção;
 - Explique brevemente as responsabilidades de cada classe/package e as principais associações, agregações e dependências;
 - Indique quais classes são parte do 'modelo' e quais pertencem à 'visualização'.
3. Padrões de software:
 - Breve descrição do padrão;
 - Razão da sua seleção nesse contexto concreto;
 - Mapeamento entre os elementos teóricos do padrão (participantes) e as classes/interfaces concretas do projeto;
 - Diagrama ou trecho de código ilustrativo que facilite a compreensão desse mapeamento.

4. Refactoring:

O relatório deverá documentar o trabalho de refactoring realizado durante a fase 3, incluindo:

- Tabela de code-smells detetados: tipo do code-smell, número de ocorrências e técnica de refactoring aplicada;
- Para cada técnica aplicada, inclua um exemplo concreto com o código 'Antes' e 'Depois' do refactoring, acompanhado de uma explicação sucinta do ganho obtido;

5. Testes e Validação

- Descreva a estratégia de testes adotada (unitários, integração, testes manuais), frameworks utilizados (ex.: JUnit) e como executar a suíte de testes;
- Inclua resultados resumidos dos testes e alguns casos de teste relevantes (entradas, passos, saídas esperadas);

6. Guia de Utilização e Instalação

- Como compilar e executar a aplicação e instruções para executar em modo de consola ou com JavaFX.
- Parâmetros configuráveis (número de pisos, número de elevadores, lotação, tempo de transição), formato de ficheiros de entrada (se aplicável) e exemplos.

7. Contribuição dos Membros

- Resumo da contribuição de cada membro de grupo (tarefas implementadas, horas estimadas).

8. Conclusões e Trabalho Futuro

- Resumo das principais decisões de design, limitações conhecidas e melhorias futuras.

4.3 Deliverables e datas de entrega

O projeto contempla 3 fases de entrega e respetivos *deliverables*:

4.3.1 Fase 1 - Milestone (data-limite: 10 de novembro de 2025, 9h00)

Entrega mínima (projeto IntelliJ IDEA com Maven):

1. Implementação inicial do modelo do simulador (classes que representam Edifício, Elevador, Piso, Passageiro, etc.). A implementação não deve conter elementos da interface gráfica;
2. Testes unitários que validem o comportamento básico das classes do modelo (ex.: geração de passageiros, embarque/desembarque, algumas estatísticas);
3. Documento README.md com: descrição das estruturas de dados e interfaces públicas, **diagrama UML de classes**, instruções para compilar/executar em modo consola, e resumo do trabalho efectuado;
4. Mockup da interface gráfica (pode ser imagem ou wireframe) incluído no README.md, com indicação dos componentes visuais que irão mapear os requisitos;
5. Plano de trabalho (secção do documento README.md) contendo a lista de tarefas para as fases 2 e 3 e a atribuição de cada tarefa a um membro do grupo.

O deliverable será apresentado em 10 minutos por cada grupo; a apresentação conta para a nota (intervalo [0, 5]).

4.3.2 Fase 2 - Projeto Funcional (data-limite: 9 de dezembro de 2025, 9h00)

Entrega mínima:

1. Aplicação funcional que resolva o problema descrito na Secção 2 e satisfaça integralmente as funcionalidades obrigatórias descritas na Secção 3.2;
2. Implementação dos dois modos de visualização: modo consola (texto) e modo gráfico com JavaFX; ambos os modos devem expor a mesma lógica de modelo por trás (e.g., separação MVC, MVP ou uma qualquer arquitetura em camadas);
3. Suíte de testes atualizada e demonstração de cenários de simulação;
4. README.md atualizado com instruções de execução para ambos os modos (com exemplos de comando e parâmetros);

Apresentação de 10 minutos para demonstração das funcionalidades implementadas; durante a apresentação serão registadas as funcionalidades funcionais para efeitos de avaliação.

4.3.3 Fase 3 – Entrega Final (data-limite: 9 de janeiro de 2026, 23h00)

Entrega mínima:

1. Aplicação gráfica (Projeto IntelliJ IDEA) que cumpra todas as funcionalidades descritas na Secção 3 (principais e adicional);
2. Pasta `javadoc` com a documentação HTML gerada;
3. Relatório completo conforme Secção 4;

⚠ Nesta fase só são permitidas alterações do tipo refactoring ao código entregue na fase 2, e a implementação de uma única funcionalidade adicional que será divulgada posteriormente. Adições de novas funcionalidades não autorizadas não serão contabilizadas.

A entrega final será alvo de discussão oral com o grupo.

5 Tabela de Cotações e Penalizações

Um projeto só será considerado “funcional” se permitir simular um edifício virtual, que é o propósito do projeto proposto. Caso contrário, não será avaliado na época de avaliação atual.

Em traços gerais a cotação total do projeto envolverá o somatório das cotações parciais dos *deliverables* Milestone e Projeto Final, segundo as seguintes ponderações:

- 25% – Milestone;
- 75% – Projeto Final.

A avaliação do trabalho será feita de acordo com os seguintes princípios:

- **Estruturação:** O programa deve ser elaborado segundo os princípios da orientação a objetos, com uma boa organização de pacotes e utilizando padrões de *software* sempre que seja adequado;
- **Correção:** O programa deve executar as funcionalidades solicitadas;
- **Legibilidade e documentação:** O código deve ser escrito, formatado e comentado de acordo com o *standard* de programação definido para a disciplina;
- **Desempenho:** As funcionalidades devem ser implementadas da forma mais eficiente possível.

Todas as cotações serão ponderadas de acordo com os princípios acima descritos.

A tabela de cotações apresentada na Tabela 1 será aplicada à Fase 3 – Entrega Final. Será também aplicada a tabela de penalizações apresentada na Tabela 2.

Item	Cotação
Modelação do problema (classes principais e ADTs)	2
Estrutura base da simulação (gestão de pisos, passageiros e elevadores)	1,5
Testes unitários (validação das estruturas e operações principais)	1
Criação e gestão dos tipos de passageiros (padrão criacional)	1,5
Implementação dos algoritmos de controlo de elevadores (padrão Strategy)	2
Simulação e atualização do estado do sistema em tempo real	1
Visualização do simulador em consola	1
Visualização do simulador em JavaFX	2
Cálculo e visualização de estatísticas (distâncias, tempos de espera, etc.)	1,5
Refactoring e otimização do código	1,5
Utilização de padrões de desenho adicionais (estruturais ou comportamentais)	1
Implementação de funcionalidade adicional (ver secção 5.3)	1
Utilização do versionamento Git	1
Relatório e Documentação (JavaDoc + relatório técnico)	3
TOTAL	20
Bónus – Funcionalidade extra (opcional e ao critério do grupo)	1

Tabela 1 - Tabela de cotações.

Descrição	Penalização (valores)
Existência de <i>code-smells</i>	0,5 por cada <i>code-smell</i> (até 3 valores)
Não utilização de padrões de software	1 por cada padrão omissos (até 4 valores)
Atraso na submissão	0,5 por cada meia hora de atraso.

Tabela 2 - Tabela de penalizações.

(fim de enunciado)